

Lecture 26: End Semester Review

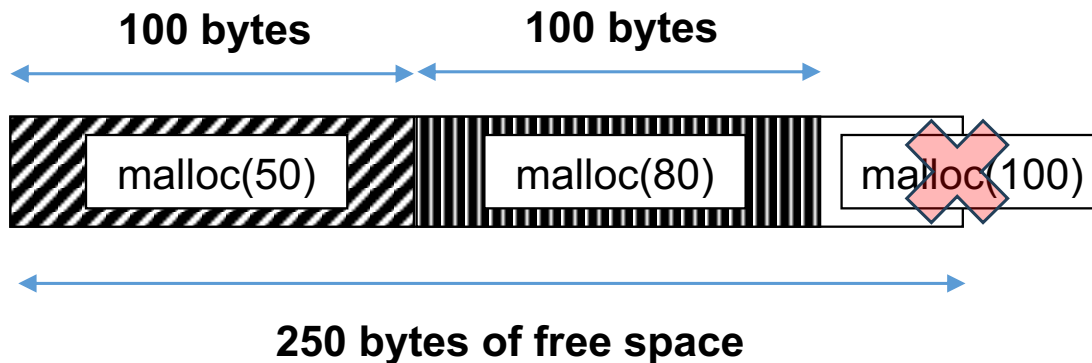
Vivek Kumar

Computer Science and Engineering

IIIT Delhi

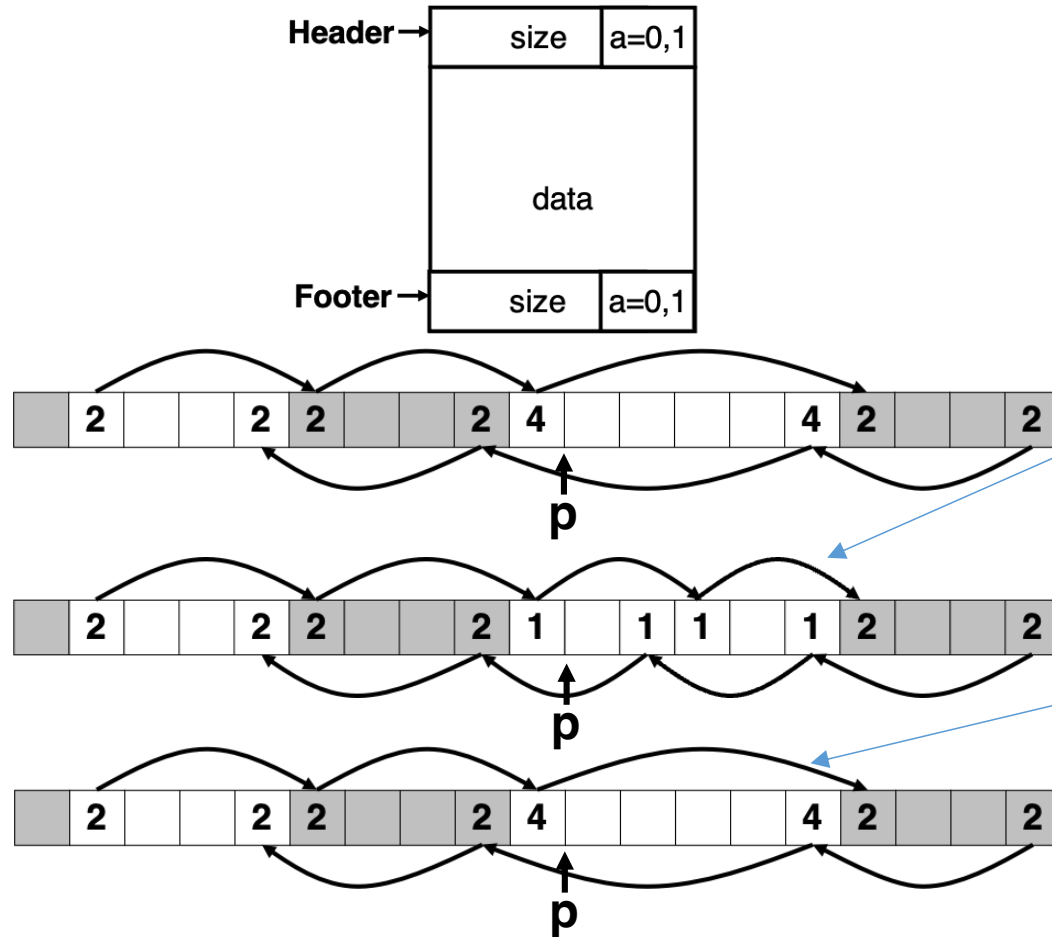
vivekk@iiitd.ac.in

Fragmentation



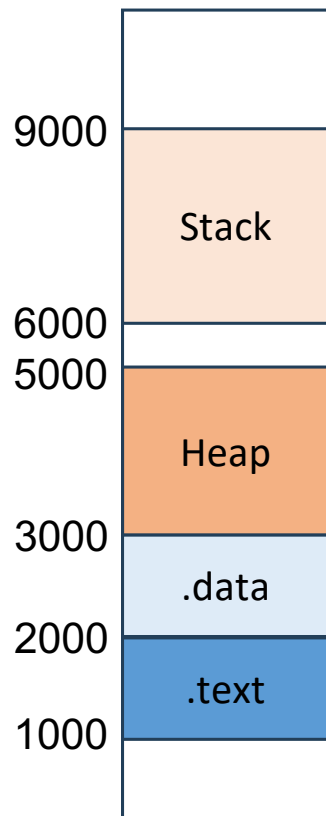
- Segments of memory can become unusable due to the result of allocation scheme
- Two types of fragmentation
 - External fragmentation
 - Memory remains unallocated
 - Internal fragmentation
 - Memory is allocated but unused
 - Fixed allocation sizes

Malloc using Implicit List



- Each memory block has a header/footer
- Blocks connected as linked list
- Block splitting during malloc
 - E.g., `p=malloc(1)`
- Block coalescing during free
 - E.g., `free(p)`

Segmentation (1/3)



- Observations
 - Each segment (stack, heap, data and text) has different memory requirements
 - Each segment can have its own (“personal”) base and bound values
 - **Segmentation**
- Each virtual address can have few bits to specify the segment they belong, and remaining bits to specify the offset into that segment

Segment Selector

Offset
- As the base and bound is separate for each segment, now it is not a requirement to have a contiguous chunk of physical address space for the process’s address space
 - Better space management and reduction in fragmentation
- Can we store the segments and their base/bound values in a MMU table?
 - Yes, but it would require **iterating** through the MMU table entries [$O(n)$]
 - **Any simple technique?**
 - **Hardware support by providing dedicated registers on each core**

Segmentation (2/3)

- x86 platform supports six segment registers per CPU

- **CS (Code Segment)**
- **SS (Stack Segment)**
- **DS (Data Segment)**
- ES, FG, and GS
 - General purpose



Base	Bound	Type	Prv
2000	3000	RE	U
4000	7000	RW	U
900	1900	RW	U

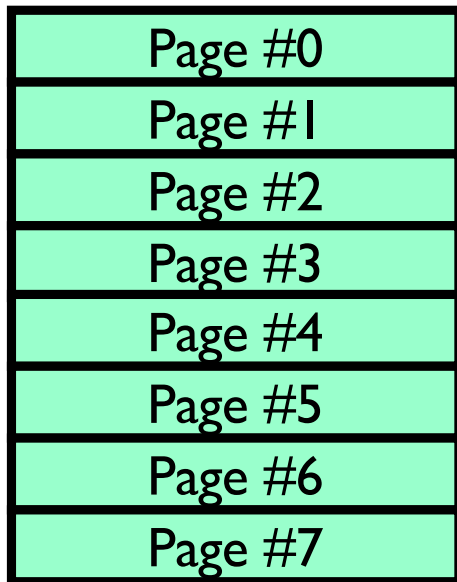
- Global Descriptor Table (GDT) that stores entries for segments from different processes
 - Base-bound pair, privilege mode for this physical memory segment (kernel vs user space), memory access type (read/write/executable), etc.
- GDT is stored in RAM
- Each CPU has a dedicated register (GDTR) that holds the starting address of this GDT

Segmentation (3/3)

- GDT requires **extra operations for each virtual memory accesses**
 1. Load segment selector from the virtual address
 2. Load GDT **offset** by looking into segment register
 3. Load GDTR and add the above **offset** to get index
 4. Load base and bound values stored at that index
- Each segment register has a visible and a hidden part
 - Visible part is programable and it is an index into the GDT for that particular segment register
 - The hidden part is non-programable and it stores the base, bound (a.k.a limit) and access information for each segment register (cache)
 - It is cached automatically the first time a GDT index is accessed
 - No need to access GDT until a context switch is done
 - No extra CPU cycles wasted for accessing GDT in VA to PA translation

Visible Part	Hidden Part	
Segment Selector	Base Address, Limit, Access Information	
		CS
		SS
		DS
		ES
		FS
		GS

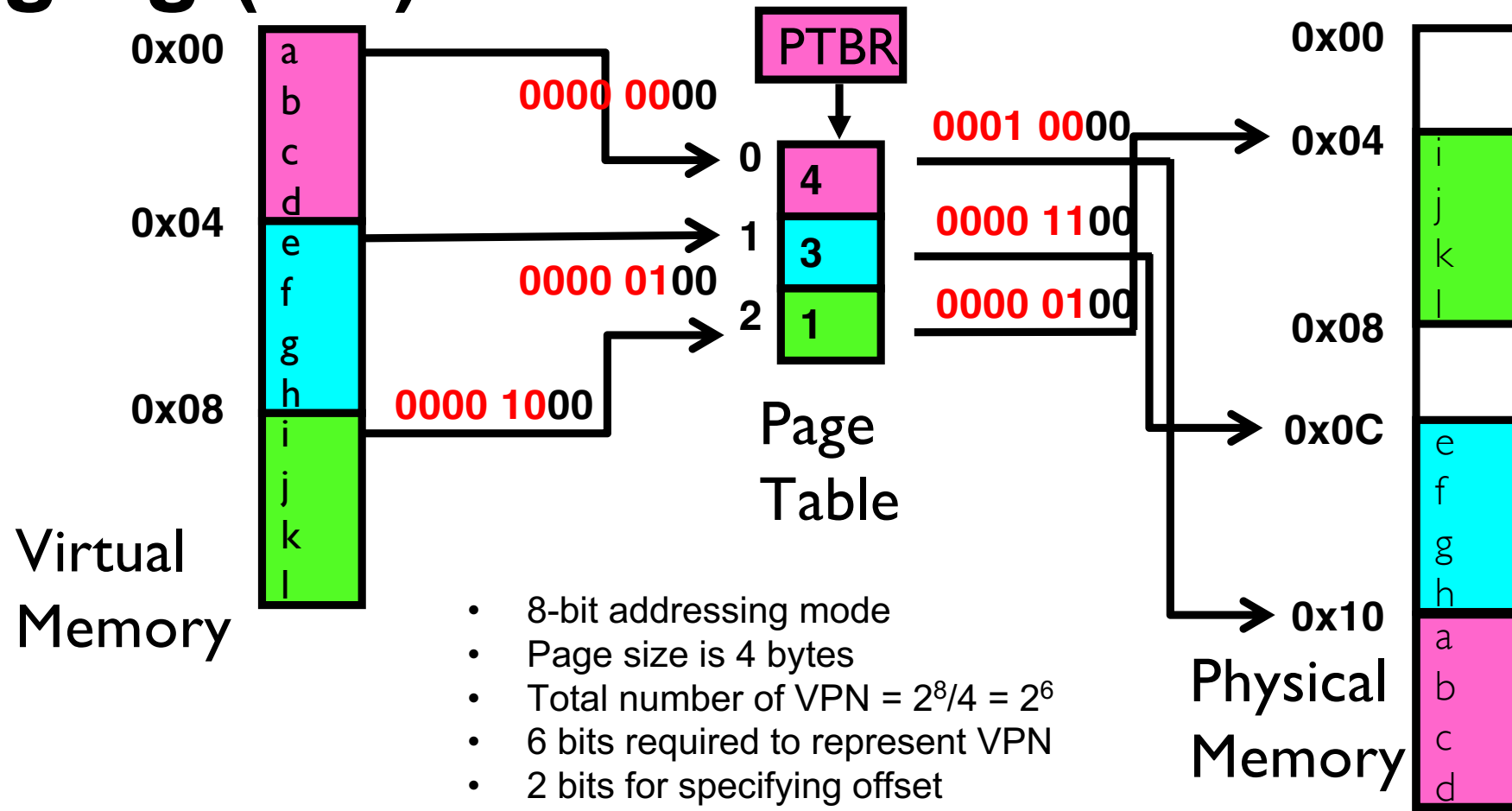
Paging (1/4) ●



Physical Memory

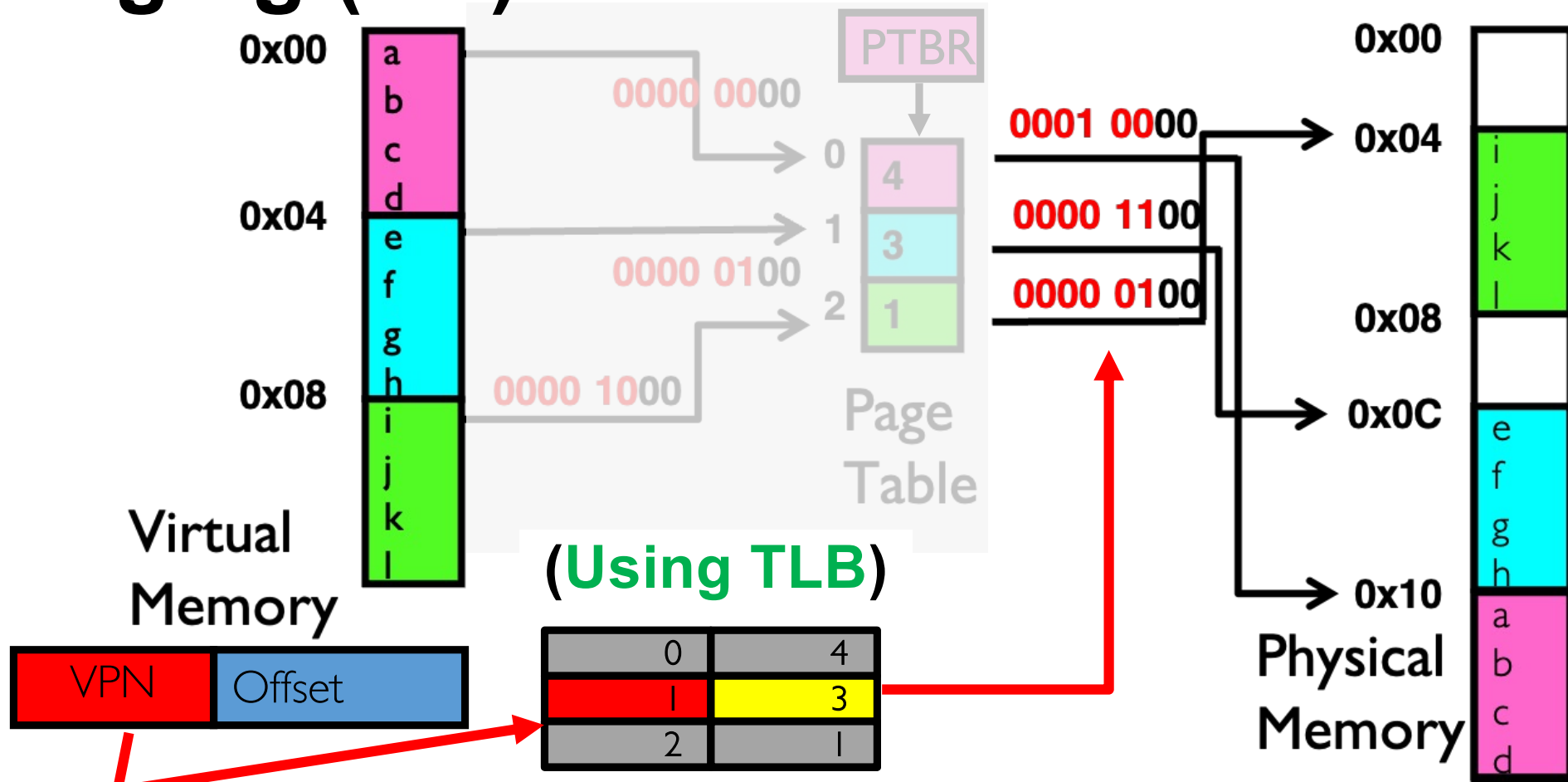
- Segmentation is easy to implement but it has major drawbacks because segments could have variable sized memory allocation
 - Can lead to **external** fragmentation
 - To allocate space for a new process, segments of some other process has to be swapped to the disk (huge overhead)
- Paging solves both the above issues by dividing the available memory into small and fixed size blocks (pages)
 - The memory can then be viewed as an array of fixed sized slots
- Default page size on Linux is 4096 bytes (4KB)
 - Total number of pages in **32-bit** addressing mode is $2^{32}/4\text{KB}$, i.e., 2^{20}
 - Total number of pages in **64-bit** addressing mode is $2^{64}/4\text{KB}$, i.e., 2^{52}

Paging (2/4)

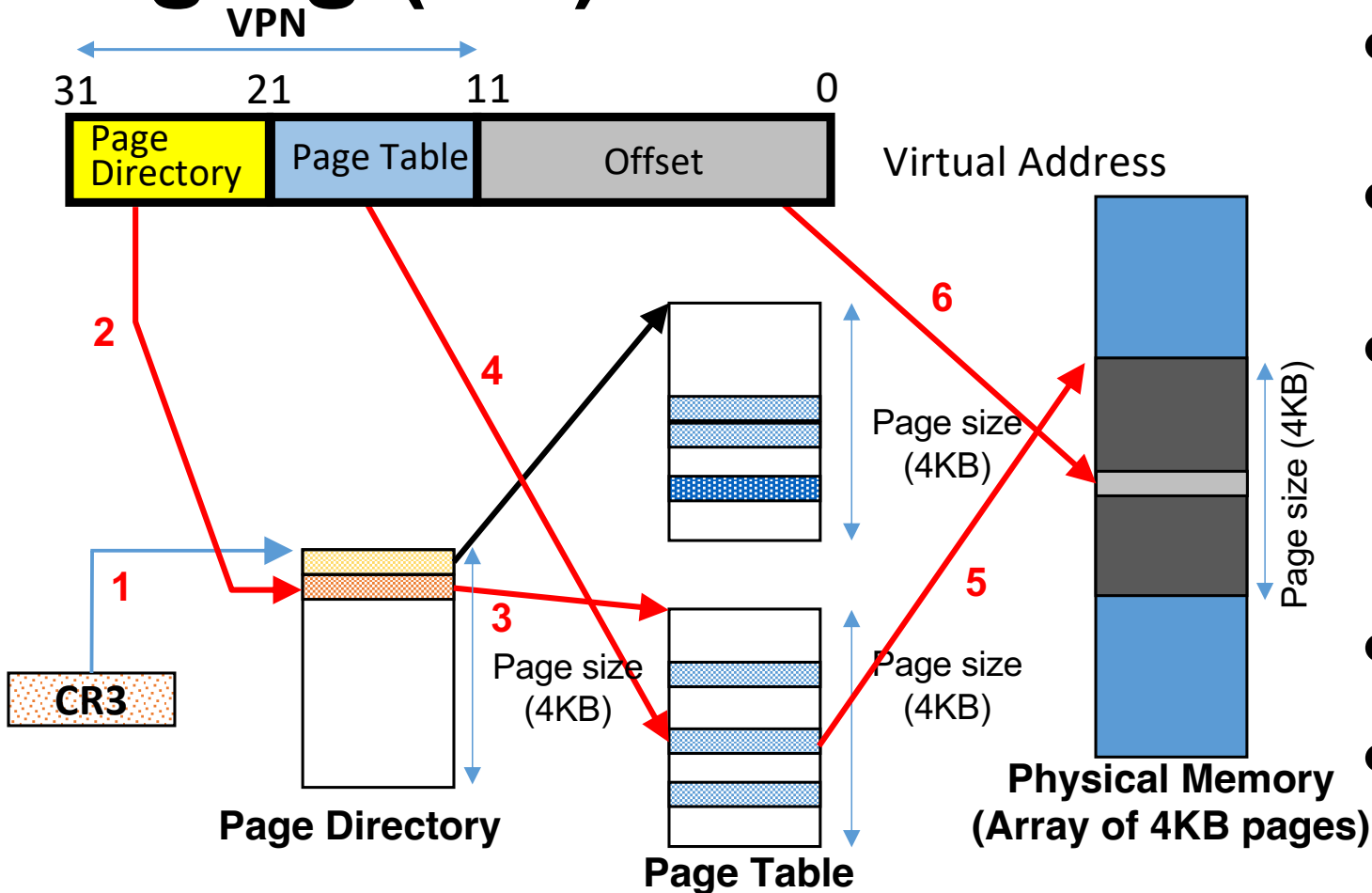


- 8-bit addressing mode
- Page size is 4 bytes
- Total number of VPN = $2^8/4 = 2^6$
- 6 bits required to represent VPN
- 2 bits for specifying offset

Paging (3/4)

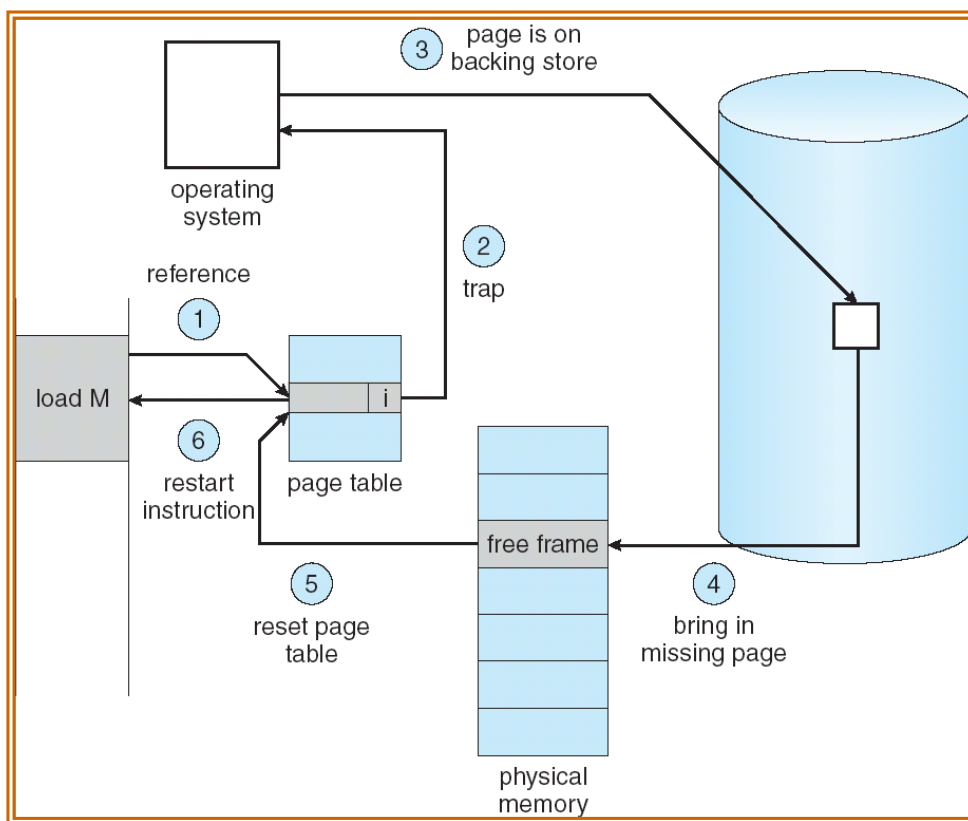


Paging (4/4)

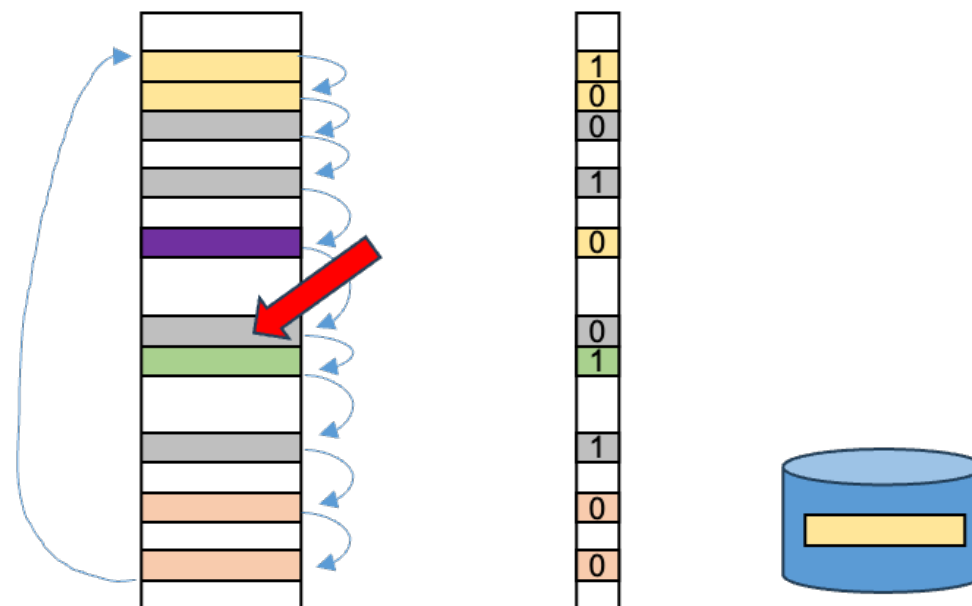


- VA stores page directory index, page table index, and address offset in PP
- CR3 register stores the starting address of the Page Directory
- Each PDE contains the Page Frame Number and a status bit
 - Status=0 if there is not a single valid entry in the PFN, otherwise status =1 (valid PFN only if status=1)
- Page Table Entry (PTE) is found from the PDE
- Physical Page Frame Number (PFN) is found from the PTE

Demand Paging



Approximating LRU using Clock Algorithm



Multithreading (1/4)

- IPC **Within** a Multicore Processor
 - **Inter-process communication in shared memory**
 - Transfer of control from user space to kernel space and vice-versa
 - **Complicated IPC mechanism for communication**
 - **OS has to reserve extra memory / resources**
 - Separate heap, stack, .text segment, etc. for each process
 - Same copy of .text segment in each process
 - **Separate page table for each process**
 - **Cost of IPC may exceed the cost of actual computation!**

Multithreading (2/4)

```

1. pthread_mutex_lock(&mutex);
2. while(task_queue_size() == 0)
3.   pthread_cond_wait(&cond, &mutex);
4. }
5. task = pop_task_queue();
6. pthread_mutex_unlock(&mutex);
7. execute_task(task);

```



Consumer(s)

```

1. pthread_mutex_lock(&mutex);
2. int queue_size = task_queue_size();
3. push_task_queue(&task);
4. if(queue_size == 0) {
5.   pthread_cond_broadcast(&cond);
6. }
7. pthread_mutex_unlock(&mutex);

```



Producer

- `pthread_cond_wait` causes the current thread to relinquish the CPU and wait until another thread invokes the **signal** or the **broadcast**
- Upon call for **wait**, the thread releases ownership of the mutex and waits until another thread signals the waiting threads

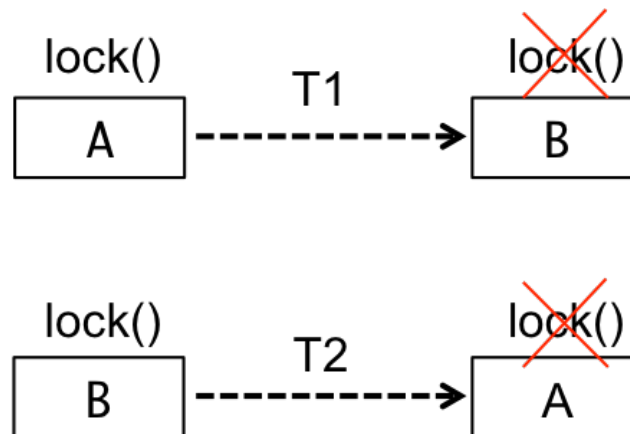
Multithreading (3/4)

```

class Transfer {
    Account A, B;
    int amount;

    void run() {
        A.lock();
        B.lock();
        A.debit(amount);
        B.credit(amount);
        A.unlock();
        B.unlock();
    }
}

```



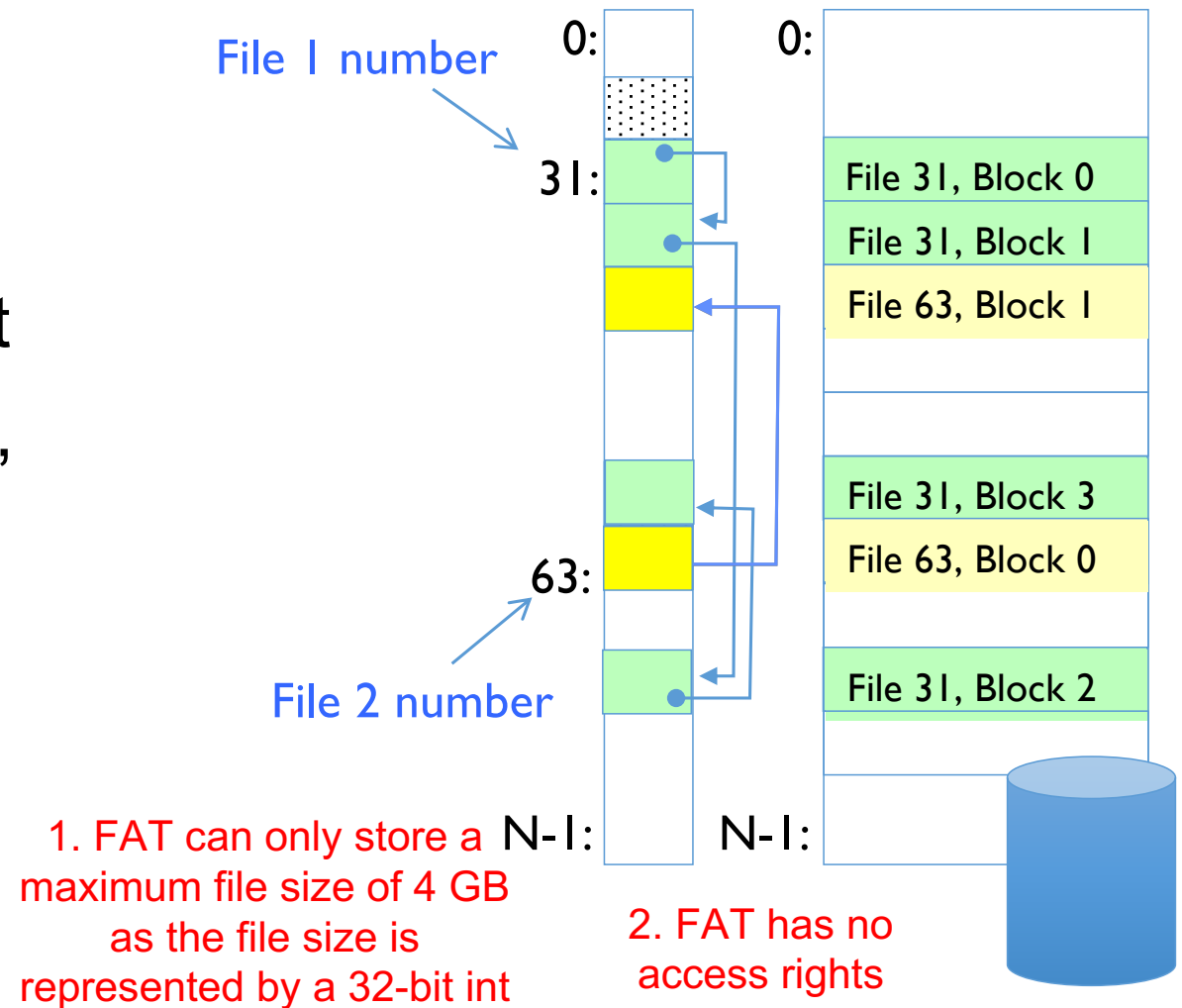
- **Deadlock** occurs when multiple threads need the same locks but obtain them in different order
 - It could be avoided by using lock ordering
 - Ensure that all locks are taken in same order by any thread

Multithreading (4/4)

- Semaphore vs. mutex variable
 - Mutex is exclusively used for achieving mutual exclusion, whereas semaphores can also be used for resource counting or signaling
 - Mutex variable is owned by a thread unlike the semaphore
 - Mutex can be unlocked only by the same thread who locked it
- Semaphore vs. condition variable
 - Semaphore can be used alone, whereas condition variable must be used along with a mutex variable
 - Unblocking multiple threads in one shot is not possible in semaphore

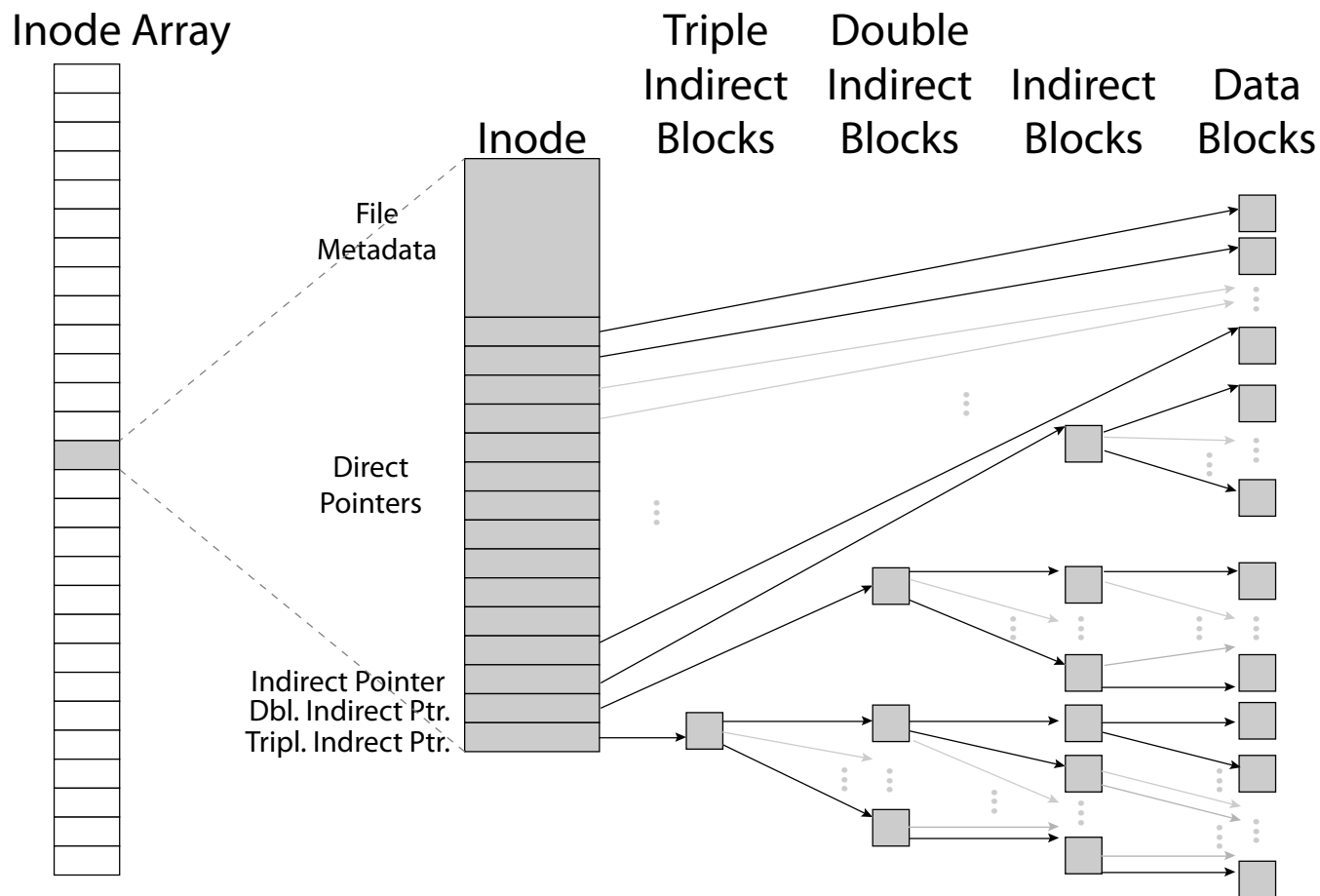
File System (1/6)

- **FAT** is a file system architecture which is really simple and robust
- It uses linked allocation, where each file is a linked list of blocks on disk
 - These blocks could be scattered any where on the disk



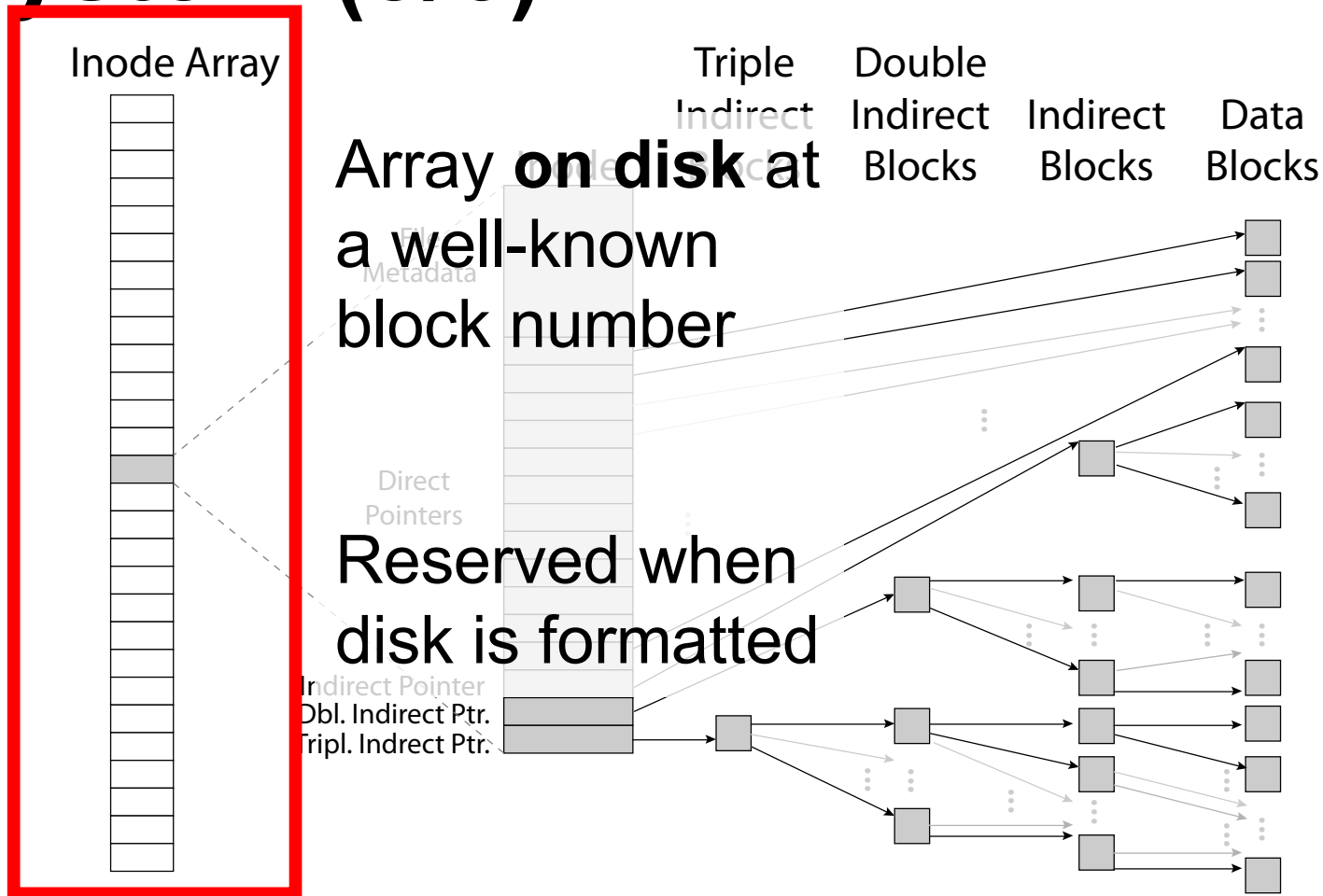
File System (2/6)

● Inodes in Unix



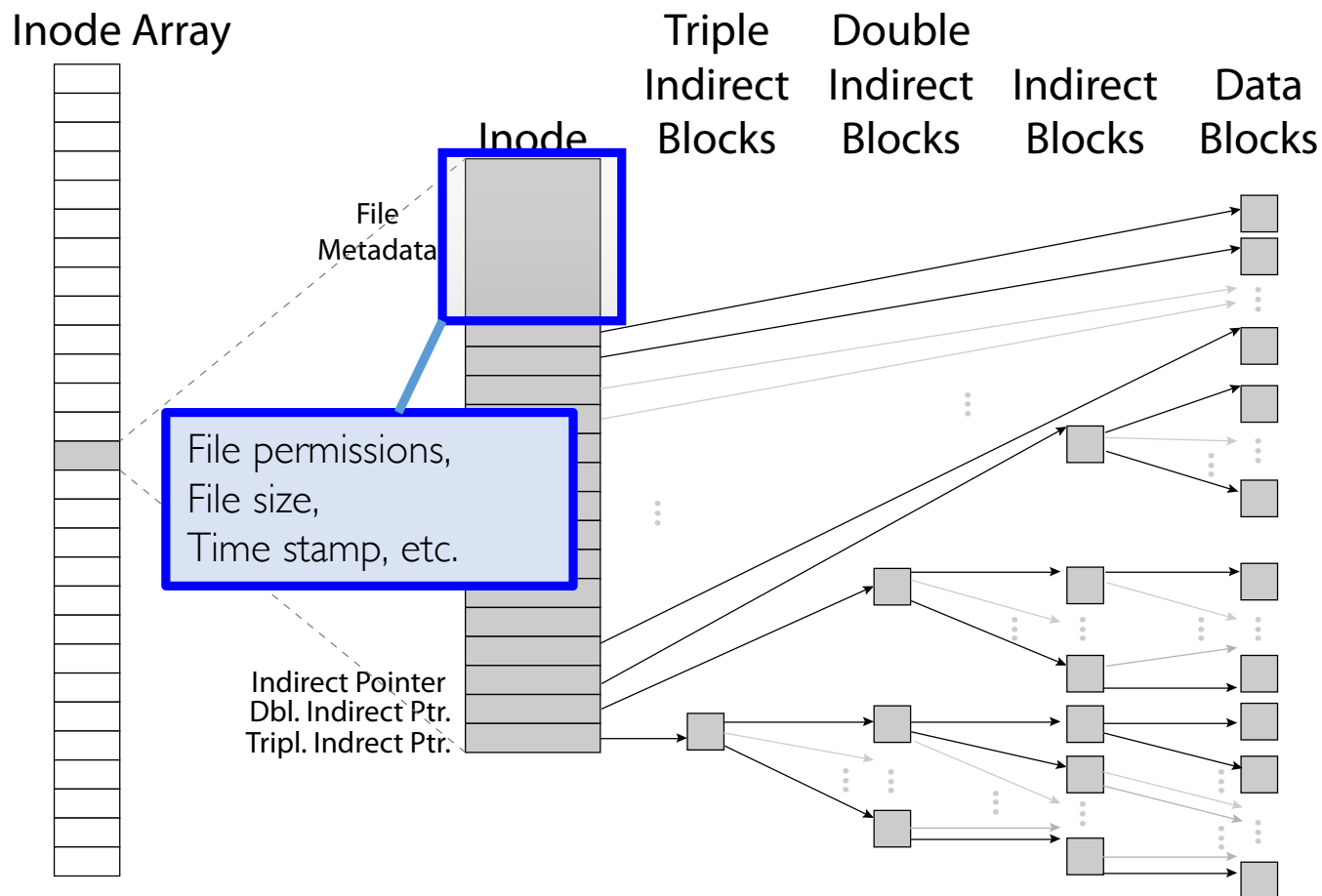
File System (3/6)

- Inodes in Unix



File System (4/6)

● Inodes in Unix

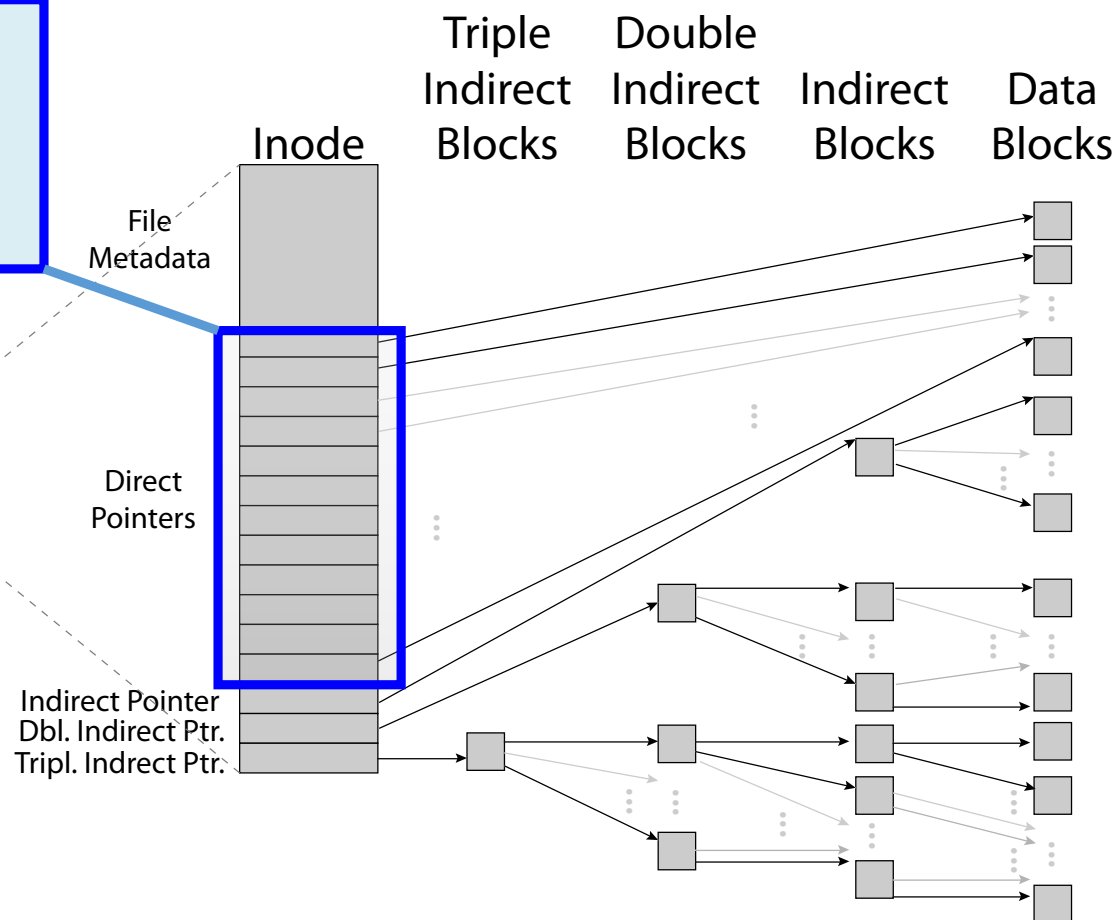


File System (5/6)

Direct pointers

4kB blocks $\Rightarrow$ sufficient
for files up to 48KB

Small files:
Level-1 has
12 pointers
direct to data
blocks (files
up to 48KB)

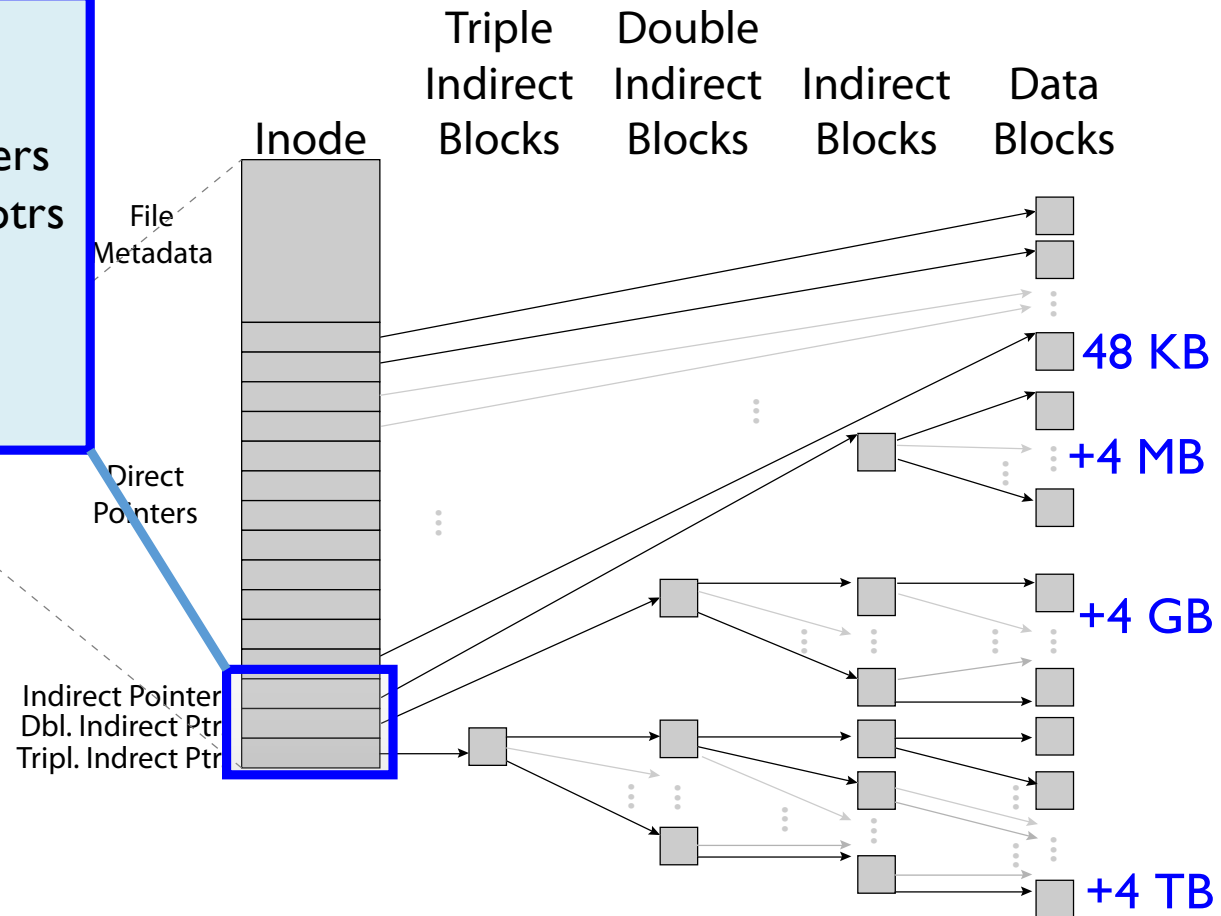


File System (6/6)

Indirect pointers

- point to a disk block containing only pointers
- 4 kB blocks $\Rightarrow$ 1024 ptrs
- $\Rightarrow$ 4 MB @ level 2
- $\Rightarrow$ 4 GB @ level 3
- $\Rightarrow$ 4 TB @ level 4

Large files:
2,3,4 level
indirect
pointers



I hope you enjoyed the course..

All the best for your exams !